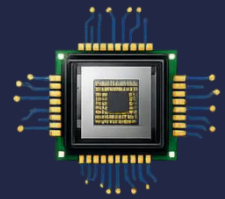


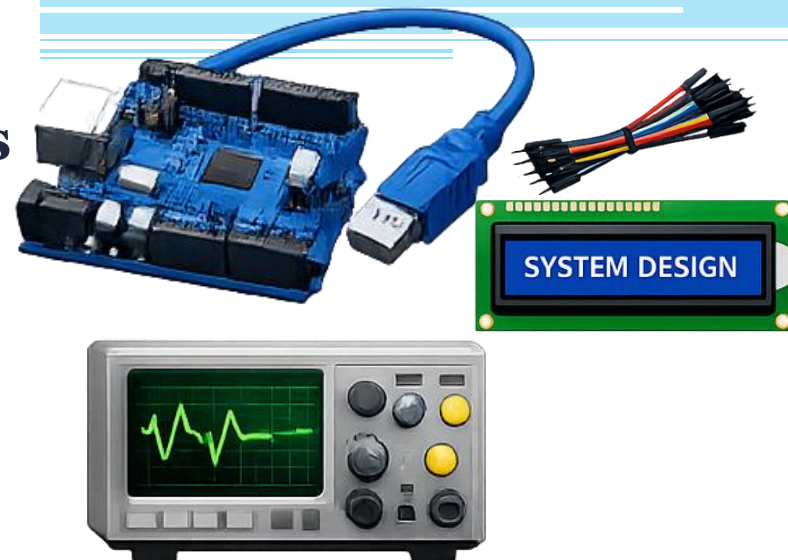
CS-301

Microprocessor- Based System Design

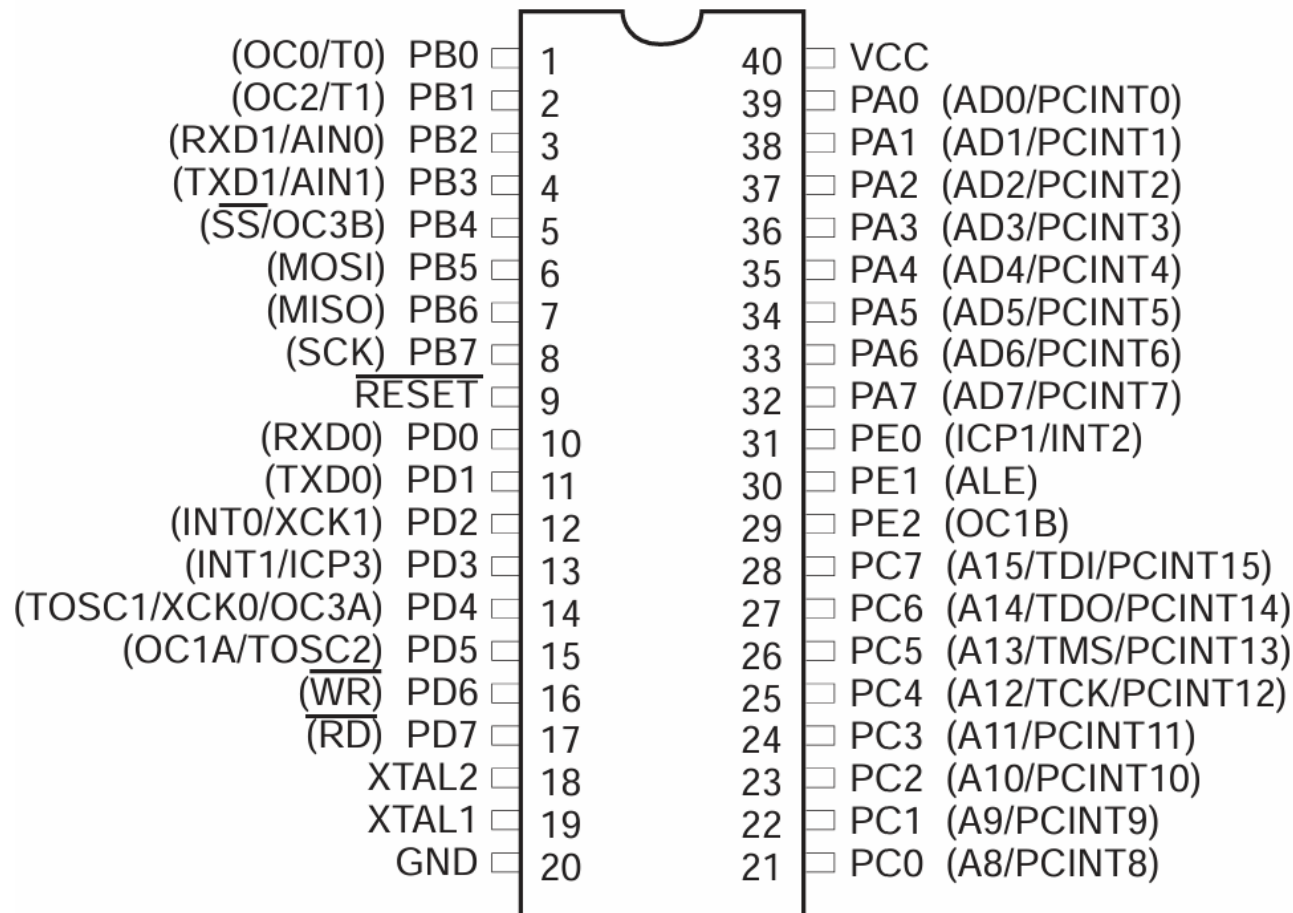


Lecture 3,4 and 5: Interfacing Microprocessors with Basic IO Devices(Switches, Push Buttons and LEDs, Seven-Segment Displays)

Course Teacher: Syeda Ramish Fatima



Pin Diagram of ATmega162

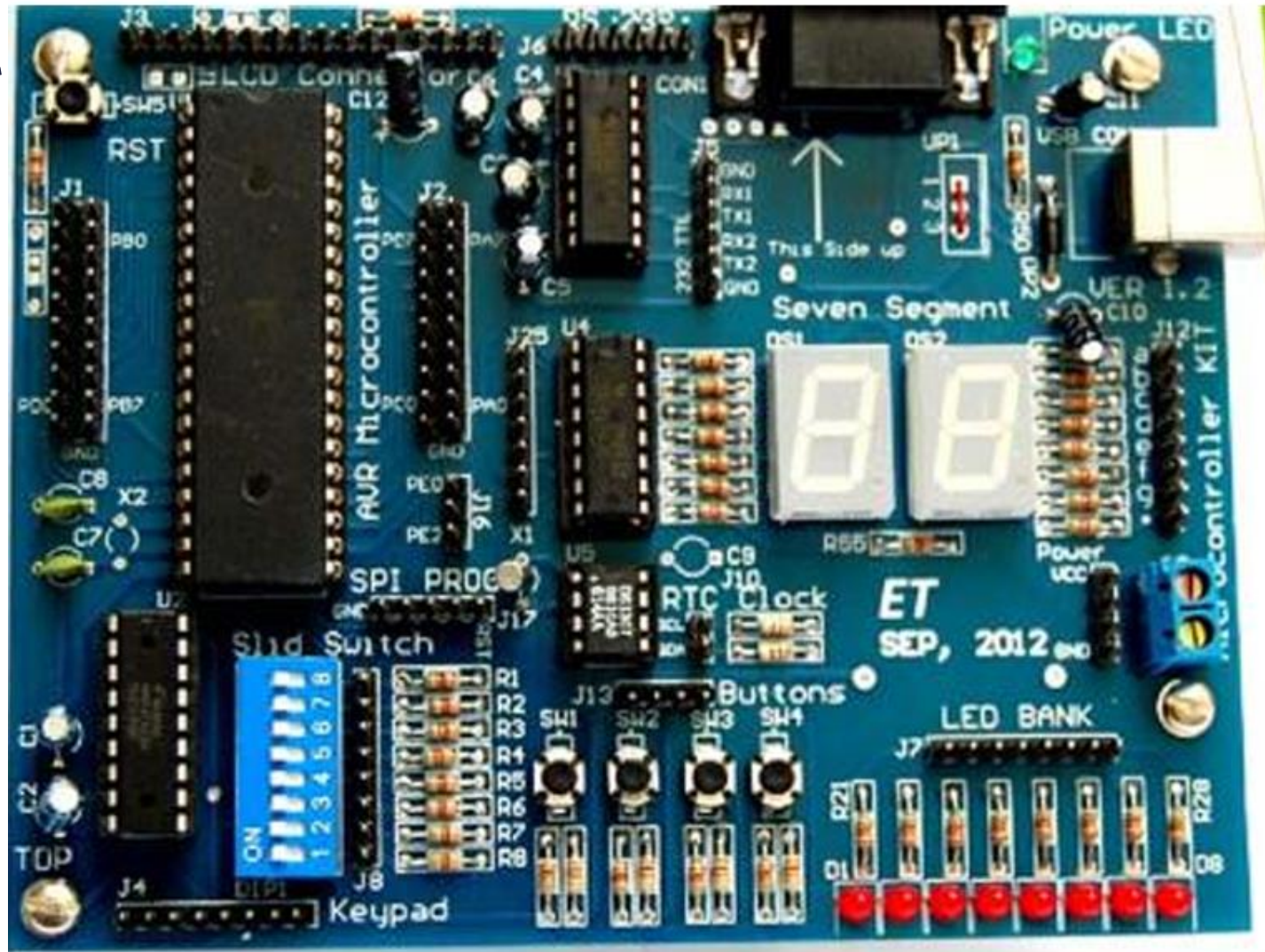


PDIP

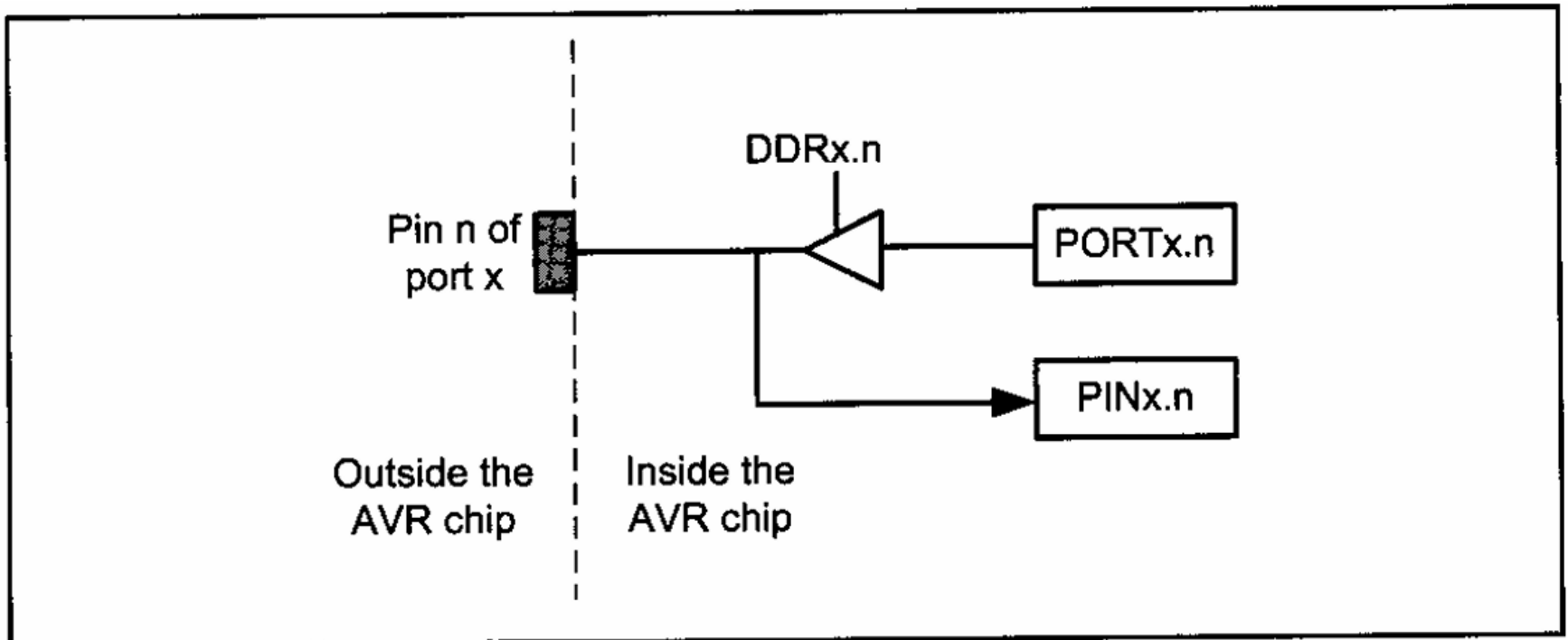
PORTS

(OC0/T0) PB0	1	40	VCC
(OC2/T1) PB1	2	39	PA0 (AD0/PCINT0)
(RXD1/AIN0) PB2	3	38	PA1 (AD1/PCINT1)
(TXD1/AIN1) PB3	4	37	PA2 (AD2/PCINT2)
(<u>SS</u> /OC3B) PB4	5	36	PA3 (AD3/PCINT3)
(MOSI) PB5	6	35	PA4 (AD4/PCINT4)
(MISO) PB6	7	34	PA5 (AD5/PCINT5)
(SCK) PB7	8	33	PA6 (AD6/PCINT6)
<u>RESET</u>	9	32	PA7 (AD7/PCINT7)
(RXD0) PD0	10	31	PE0 (ICP1/INT2)
(TXD0) PD1	11	30	PE1 (ALE)
(INT0/XCK1) PD2	12	29	PE2 (OC1B)
(INT1/ICP3) PD3	13	28	PC7 (A15/TDI/PCINT15)
(TOSC1/XCK0/OC3A) PD4	14	27	PC6 (A14/TDO/PCINT14)
(OC1A/TOSC2) PD5	15	26	PC5 (A13/TMS/PCINT13)
(<u>WR</u>) PD6	16	25	PC4 (A12/TCK/PCINT12)
(<u>RD</u>) PD7	17	24	PC3 (A11/PCINT11)
XTAL2	18	23	PC2 (A10/PCINT10)
XTAL1	19	22	PC1 (A9/PCINT9)
GND	20	21	PC0 (A8/PCINT8)

AVR Kit



Understanding the IO Port



Understanding the Pull-up Resistor

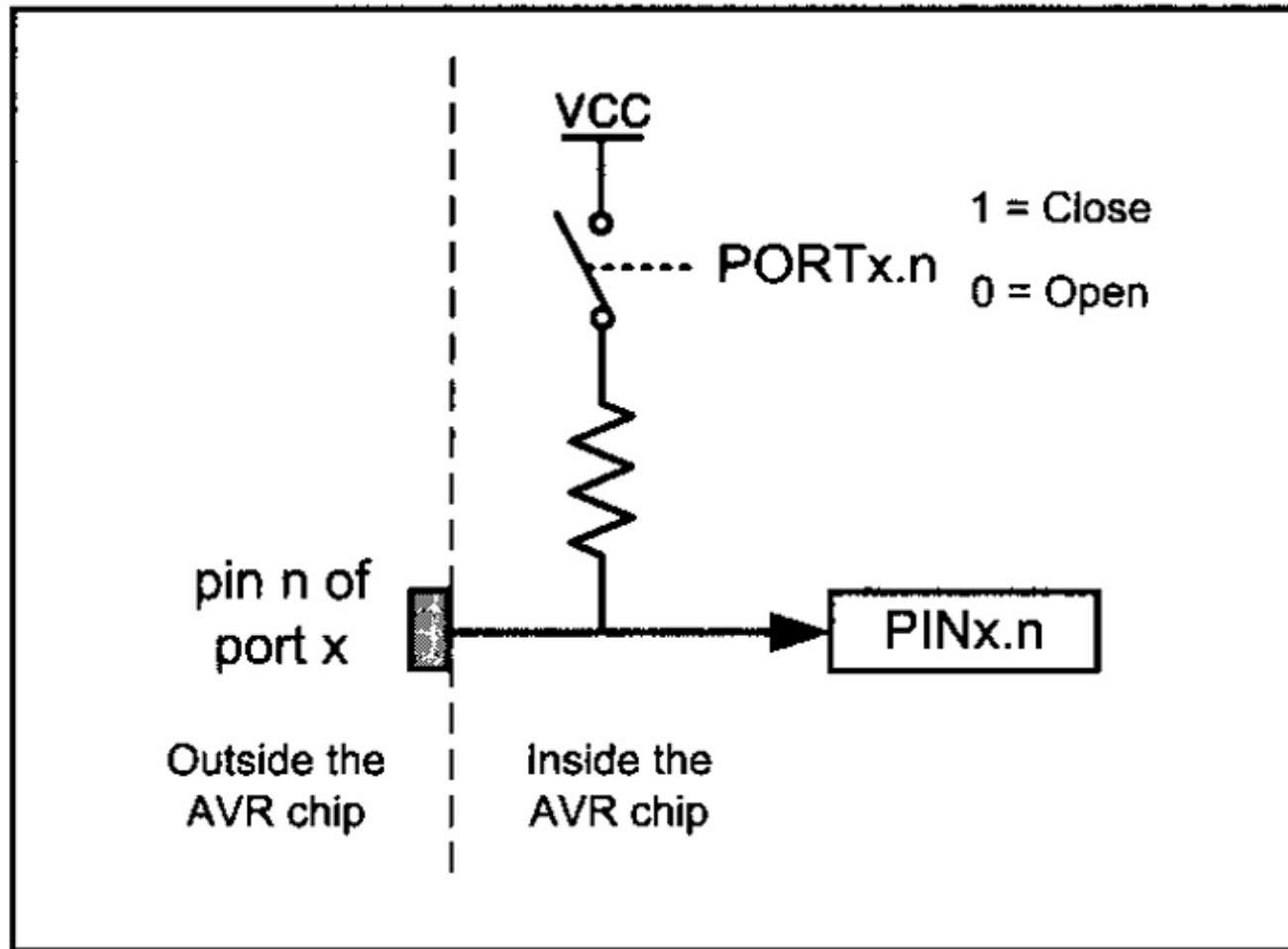
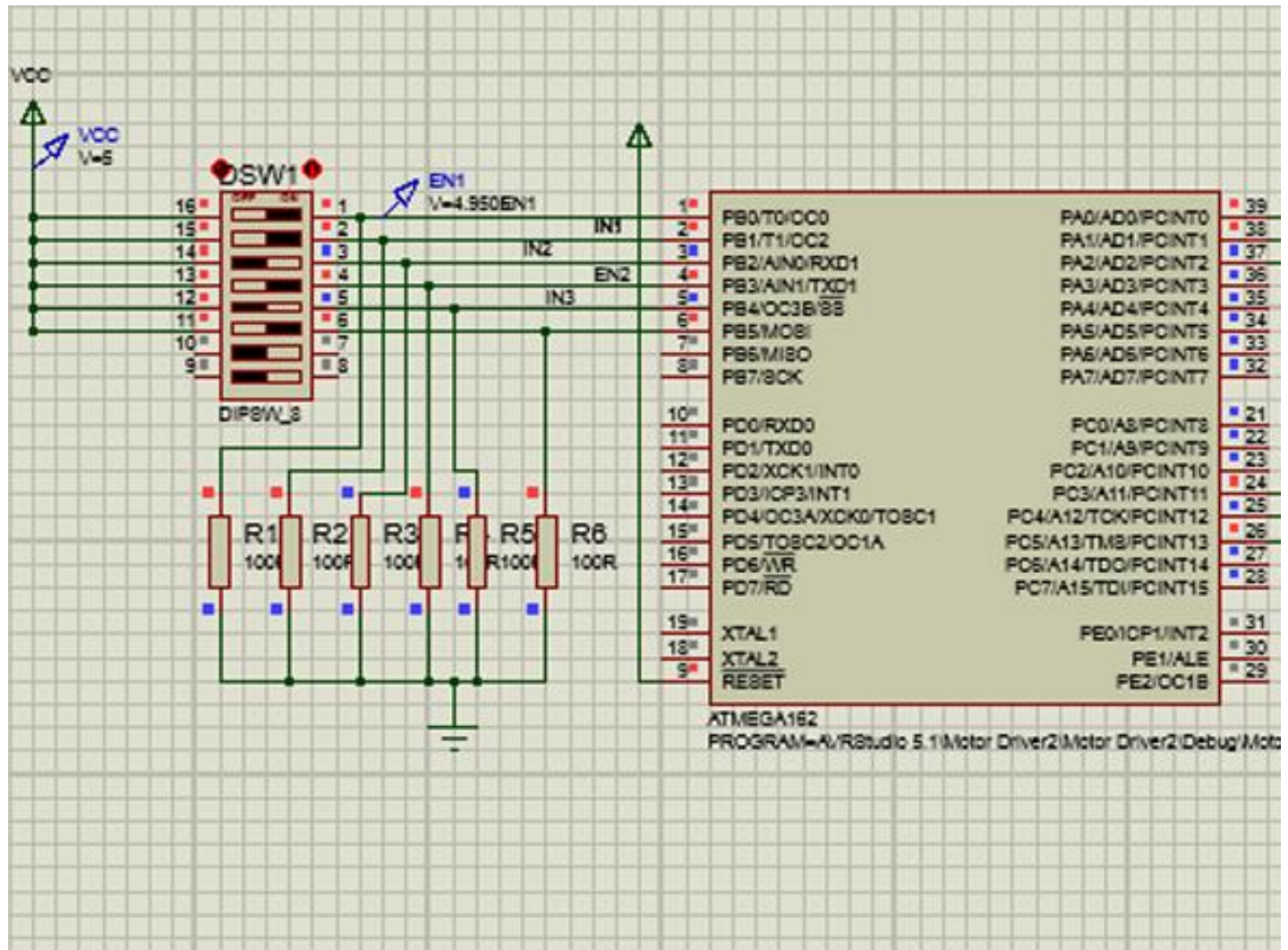


Figure 4-4. The Pull-up Resistor

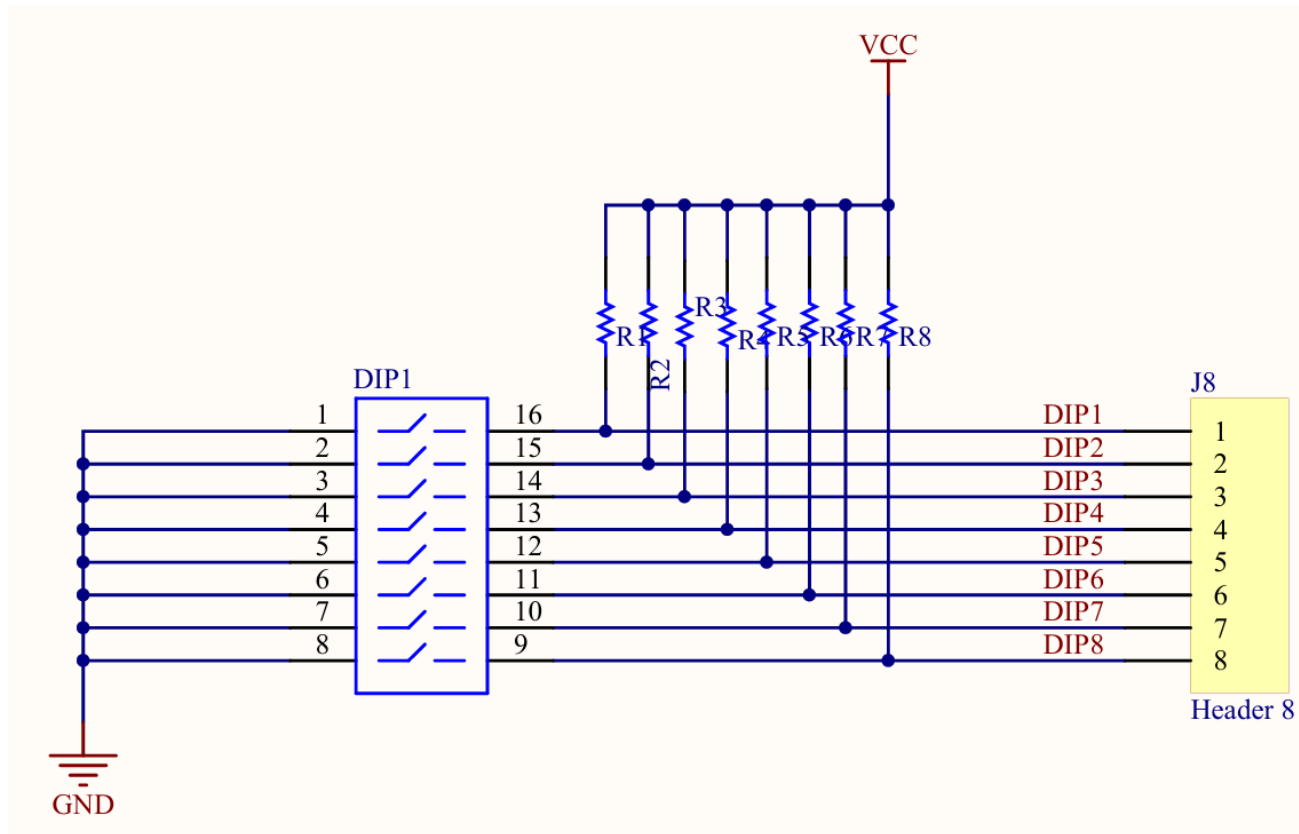
Interfacing with dip switch



Interfacing with dip switch

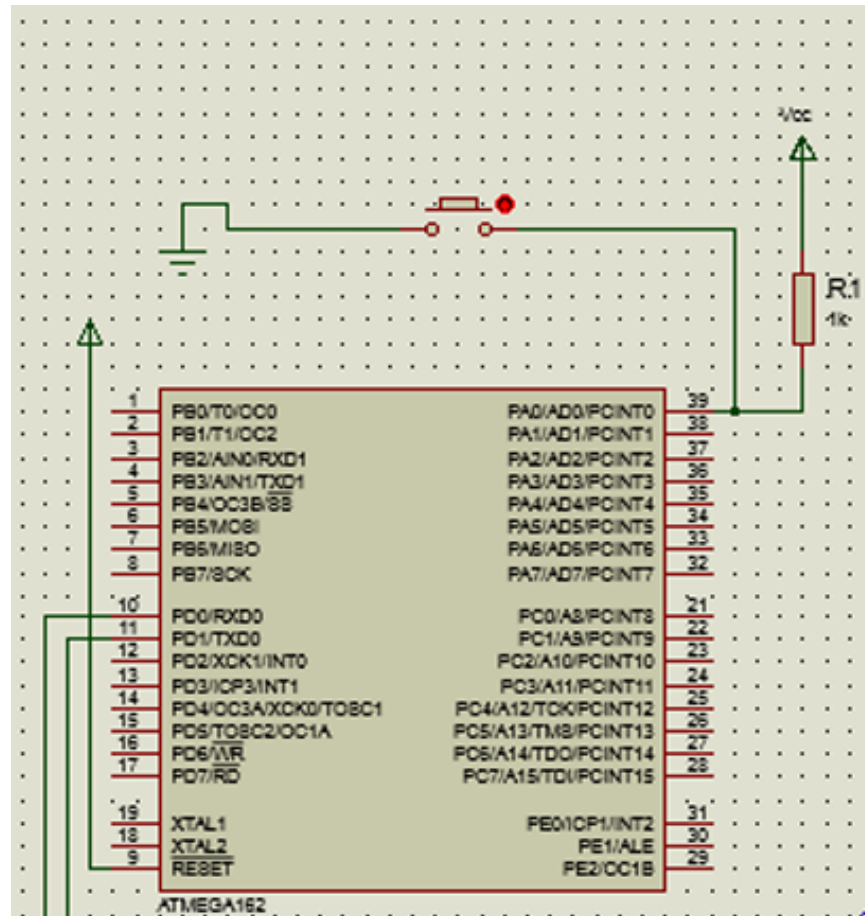
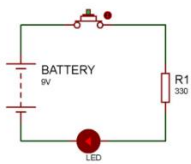
```
#include <avr/io.h>
#define F_CPU 1000000UL
#include <util/delay.h>
int main(void)
{
    DDRB=0b00000000; // set port B as input
    //since no value is being assigned to PORTB register , no pull-up register
    char value; // to read the value from input port register PINB
    while(1)
    {
        value=PINB; // PINB register is read in an 8-bit variable
    } // while ends
} //main ends
```


Interfacing with dip switch



Interfacing with push button

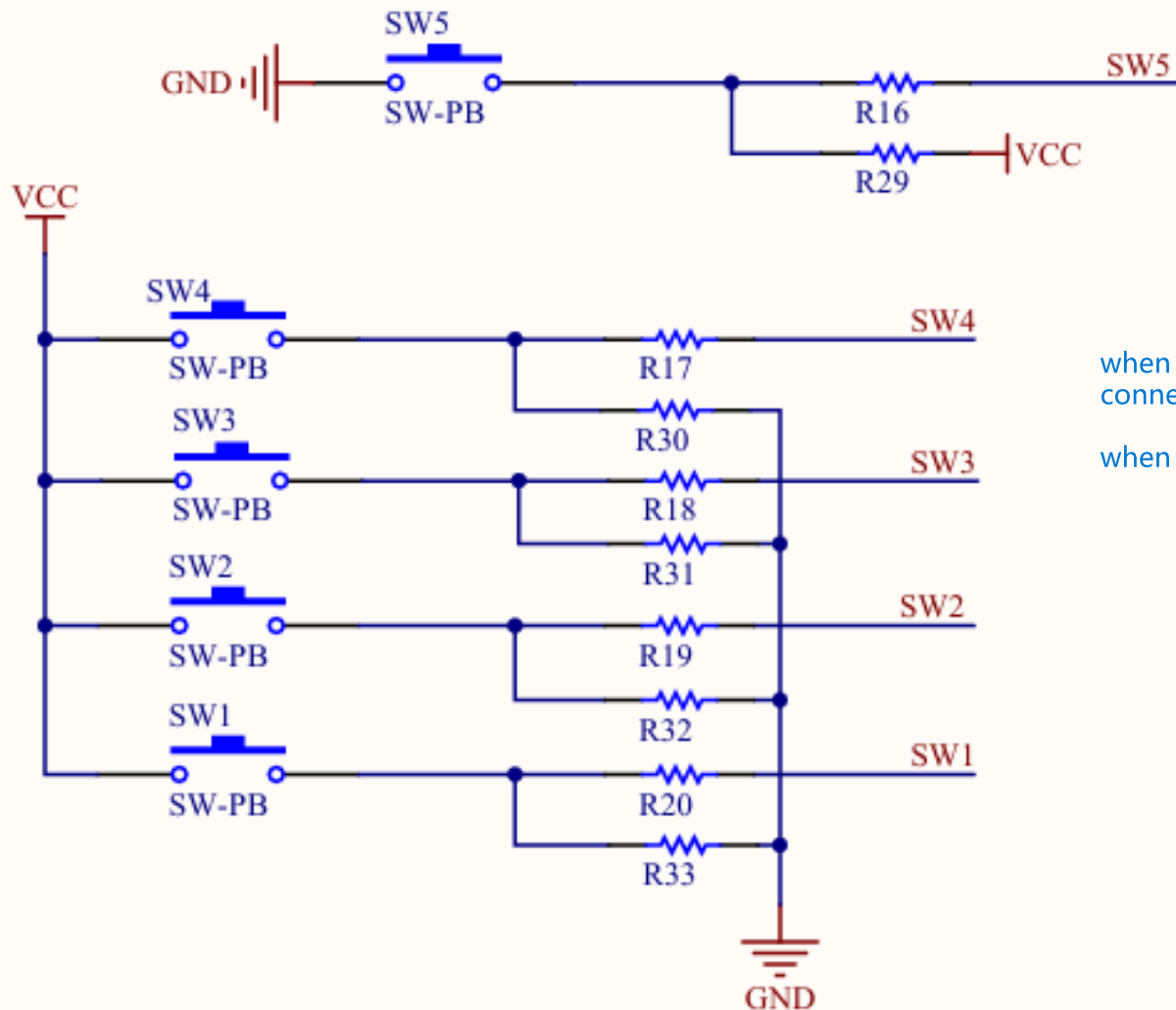
Push Button LED Circuit



Interfacing with push button

```
#include <avr/io.h>
#define F_CPU 1000000UL
#include <util/delay.h>
int main(void)
{
    DDRA=0b00000000; // set port A as input. Only DDA0 can be made 0 rest of the bits sre don't care
    //since no value is being assigned to PORTA register , no pull-up register
    char value; // to read the value from input port register PINA
    while(1)
    {
        value=PINA; // PINA register is read in an 8-bit variable but only bit 0 is important
    } // while ends
} //main ends
```

Interfacing with push button



when not connected the output will be high since, connected to vcc

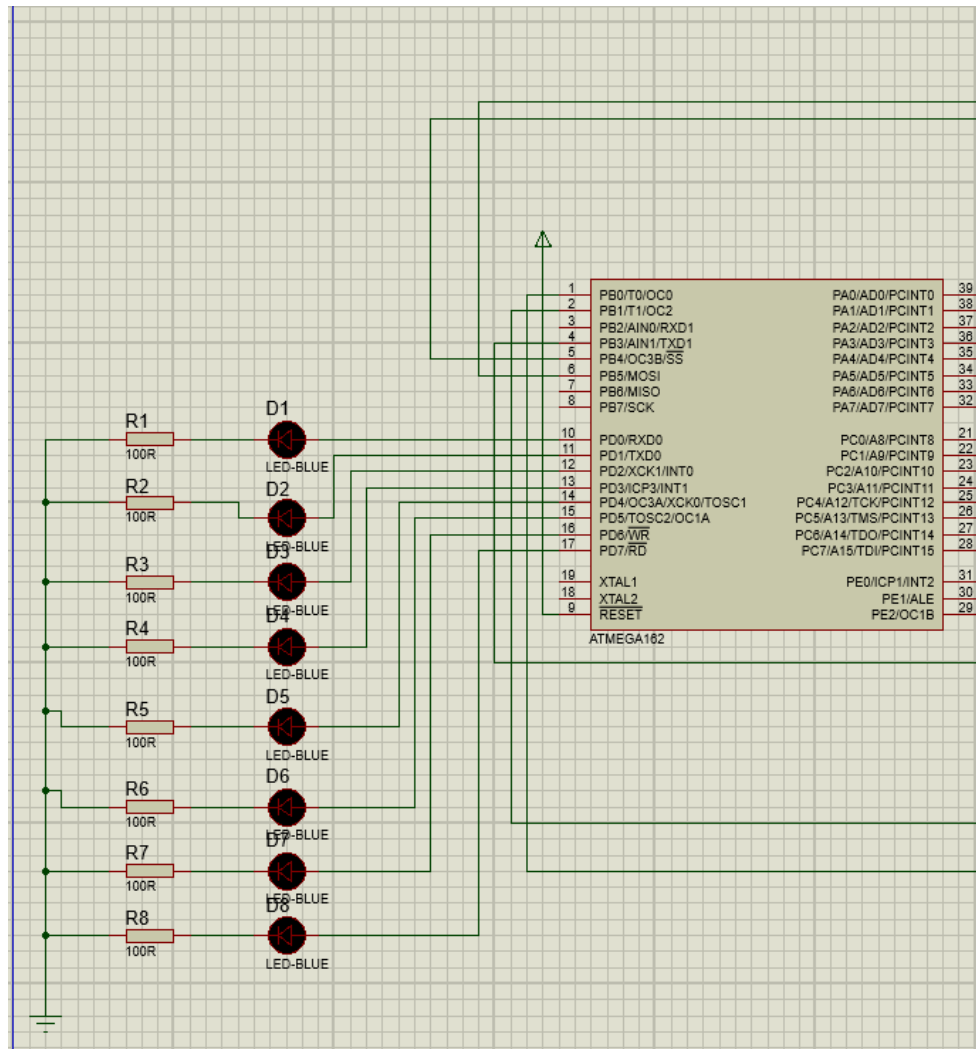
when connected, output will be low

when not connected the output will be low since, connected to gnd

when connected, output will be high



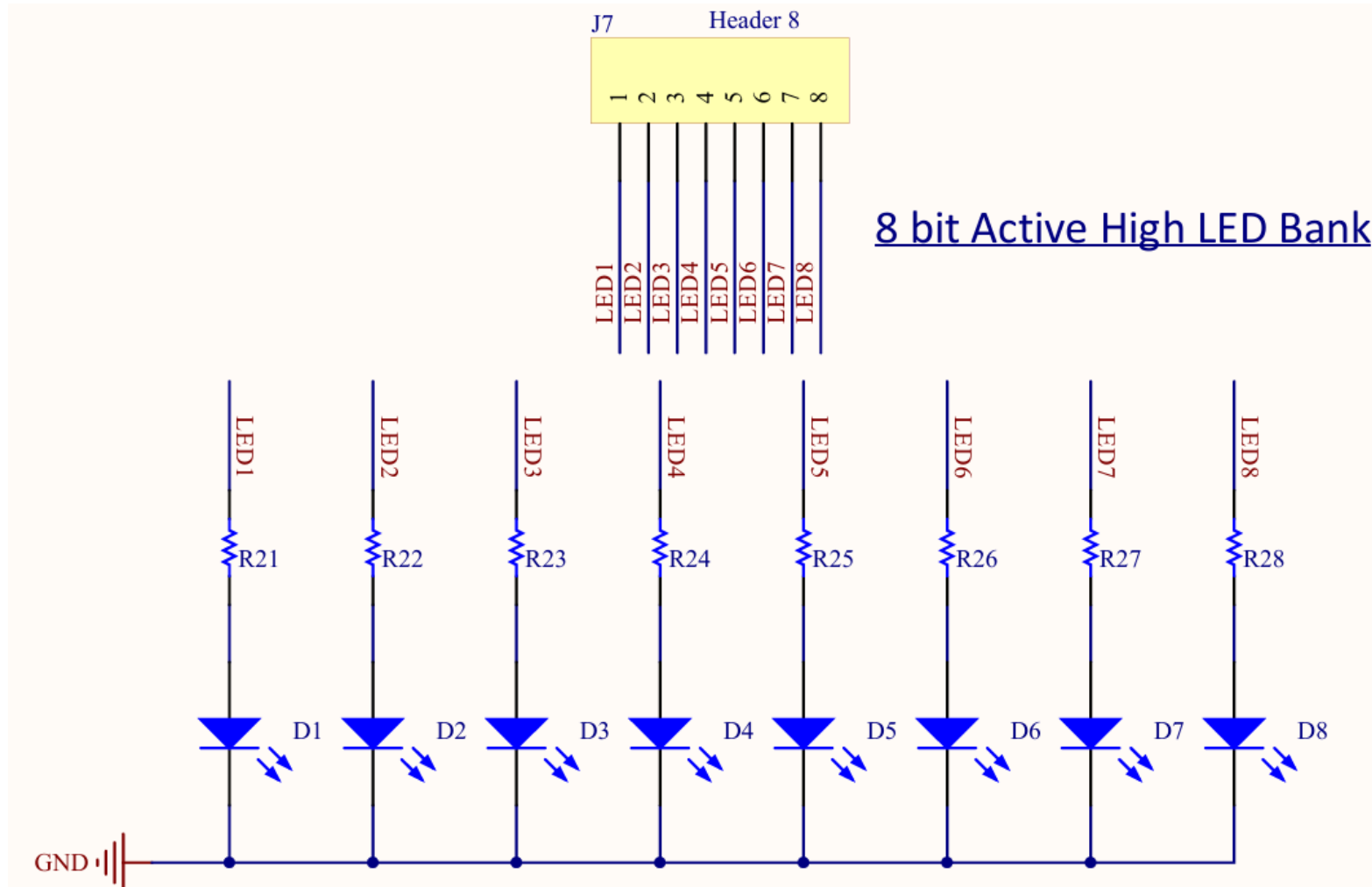
Interfacing with LEDs



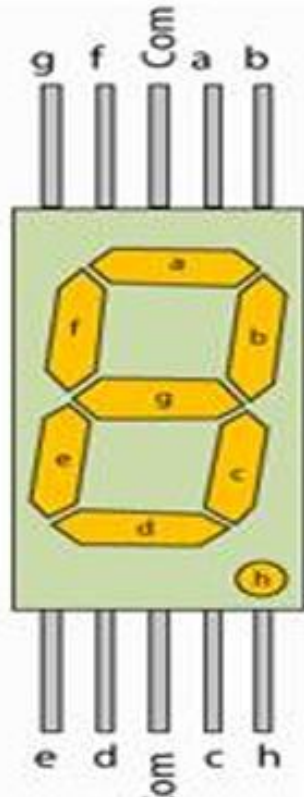
Interfacing with LEDs

```
#include <avr/io.h>
#define F_CPU 1000000UL
#include <util/delay.h>
int main(void)
{
    DDRD=0b11111111; // set port D as output.
    char value=0b10101010; // to assign to PORTD to drive LEDS
    while(1)
    {
        PORTD=value; //PORTD register is being assigned the value to drive LEDS
    } // while ends
} //main ends
```

Interfacing with LEDs

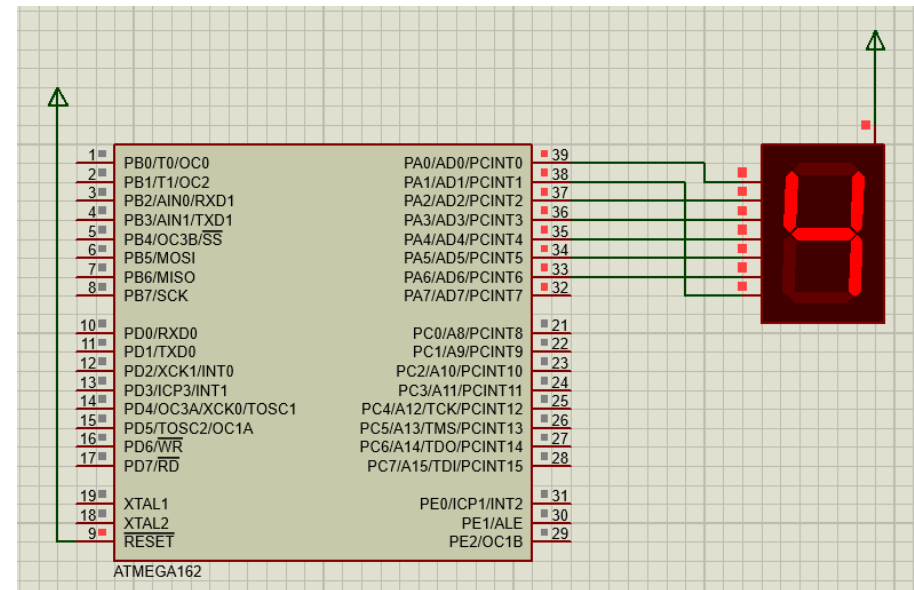
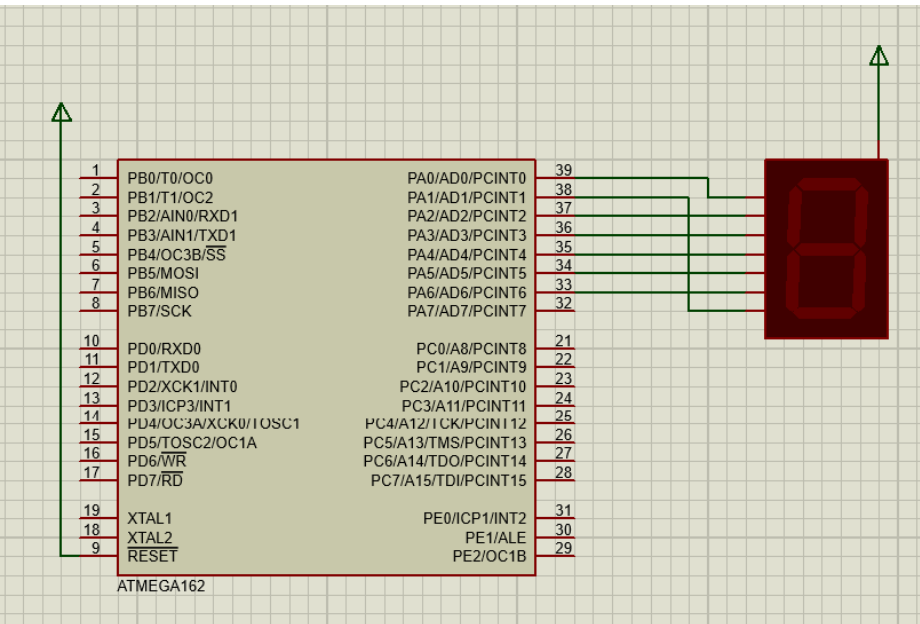


Seven Segment Displays



- 7-segement display is a ten pin device, in which 8 pins are the single terminals of eight LEDs, while the other terminal of all LEDs is connected together and named as common(Com).
- Based on common pin connection there are two types of Seven Segment Display configuration.
 - **Common Anode:** In this configuration Common pin is internally short with Vcc, to turn an LED on the respective segments pin must be connected to ground or must be written with logic 0.
 - **Common cathode:** In this configuration Common pin is short with ground; the other open terminals of LEDs need to be provided with logic high signal to glow.

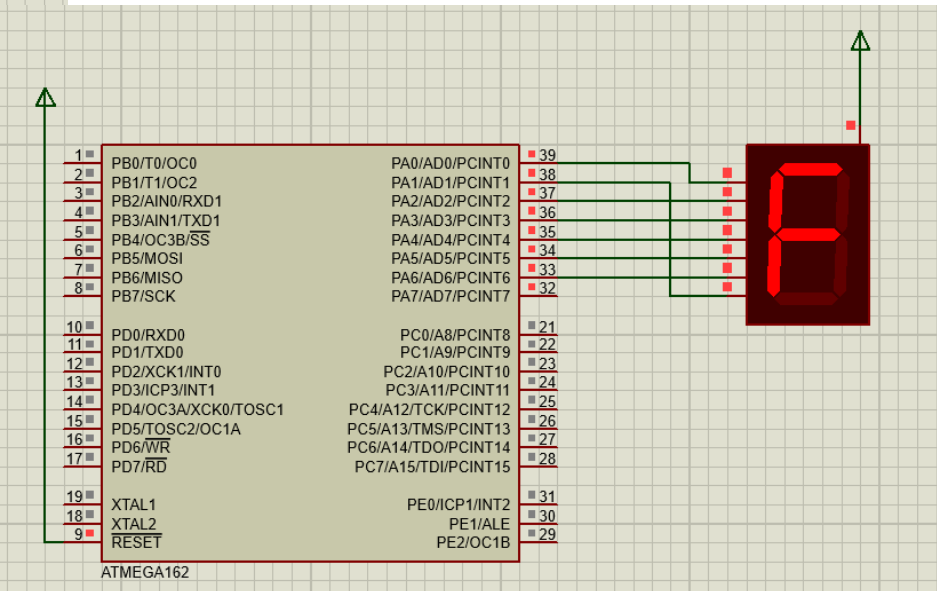
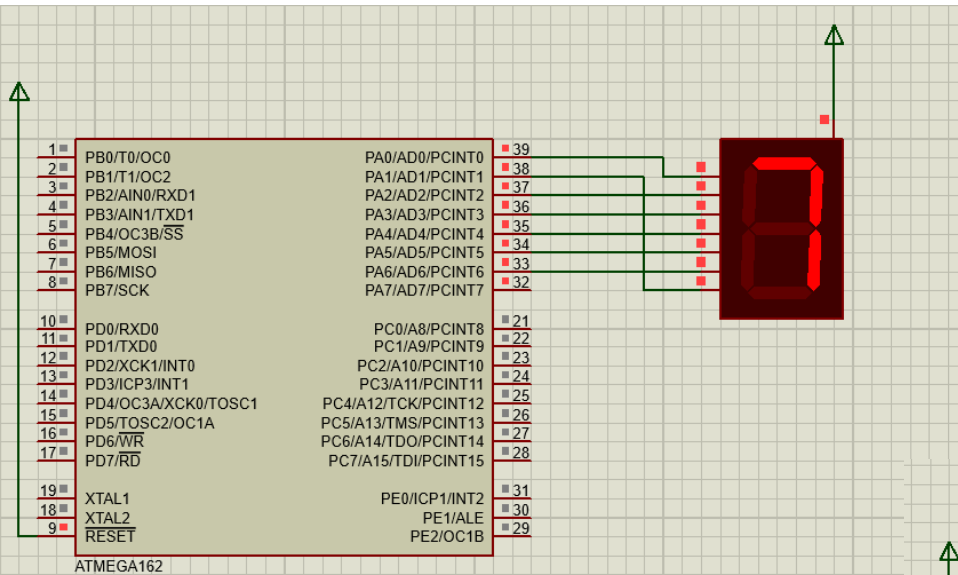
Seven Segment Displays



Interfacing with Seven segment displays(Common Anode)

```
#include <avr/io.h>
#define F_CPU 1000000UL
#include <util/delay.h>
int main(void)
{
    DDRA=0b11111111; // set port A as output.
    char value=4; // to assign to PORTD to drive seven Segment LEDS
    while(1)
    {
        PORTA=0b10011001; //PORTA register is being assigned the value to drive LEDs
    } // while ends
} //main ends
```

Seven Segment Displays



Interfacing with Seven segment displays(Common Anode)

```
#include <avr/io.h>
#define F_CPU 1000000UL
#include <util/delay.h>
int main(void)
{
    DDRA=0b11111111; // set port A as output.
    char value=4; // to assign to PORTD to drive 7-segment LEDS
    while(1)
    {
        PORTA=0b11111000; //PORTA register is being assigned the value to drive LEDs
        _delay_ms(500);
        PORTA=0b10001110;
        _delay_ms(500);
    } // while ends
} //main ends
```




- *Further Reading: Go through the datasheet of 7447 BCD-seven segment decoder IC from the datasheet provided.*